

30 — Pinned baseline index out of range

Proves · A pinned baseline index outside the valid range, and how it is handled.

Mechanics this scenario turns on

- Baseline rule: **no battery** → **the worst combination** (`count - 1`); **battery active** → **the third from worst** (`Math.Max(0, count - 3)`). It models what the ship is assumed to do today.
- `Math.Max(0, ...)` bites on short lists: with only two combinations, `2 - 3` clamps to **0** — the optimum itself — so that mode reports **zero** Level 1 savings. A battery in a mode can therefore suppress that mode's Level 1 savings; the value moves to the Battery Benefit badge instead.
- The **Assumed Configuration** table shows Transit's combinations only, in t/h. The Fuel Consumption figure above it is the annual total across **all** modes.

Panels described below · The rule · Verified against scenario 01 · Where a bad index comes from

Anything not described here — the mechanics above name the step that produced it; `00-ORIENTATION` Part 6 has the full number-to-step index.

Trust · characterisation snapshot, generated from the code. It detects change; it does NOT prove correctness. Figures marked *pending reference verification* have never been checked against anything outside the application.

Read after · scenario 15.

Scenario 15 pins a **valid** row. This pins row **99** on a five-row table.

The rule

```
requestedIndex is int index && index >= 0 && index < sorted.Count
    ? index
    : defaultIndex
```

An index that does not address the list is **ignored**, not clamped. That distinction is the whole scenario: clamping to the last row would look almost right (the default with no battery *is* the last row) and be wrong the moment a battery is active, because the battery default is the **third** highest, not the last.

Verified against scenario 01

	01 (no pin)	30 (pin = 99)
valid combinations	5	5
<code>selectedBaselineIndex</code>	2	2
baseline FOC	13 307.40	13 307.40

Byte-identical baseline block. Index 2 on a five-row list is `Count - 3` — the D1 battery rule — which confirms the fallback went to the *battery* default and not to "last row".

Where a bad index comes from

Not from the UI — the client only sends indices it rendered. It comes from **saved profiles**: pin row 4, save, then reload the profile after changing the plant so fewer combinations exist. The stored index now points past the end.

Scenario 15's card documents the client-side bug where the pin was *lost* on restore (finding 5, since fixed). This one covers the opposite hazard: a pin that survives but no longer fits.

Takeaway: silently ignoring a stale pin is the right behaviour, and it is only correct because the fallback re-derives the default rather than reusing a remembered one.

KSailCalc manual test scenarios — calculation walkthrough · regenerated 2026-08-10 · numbers verified against commit 559aef2 and unchanged by the backend/client refactors (18 golden snapshots byte-identical)